JAVASCRIPT

DOM

EVENTOS

- Eventos en atributos HTML
- Eventos en propiedades Javascript
- addEventListener()
- Múltiples listeners
- removeEventListener()
- Objeto Event
- Metodo PreventDefault

PROFESORA:
ELIZABETH GISELA OVIEDO LUNA

EVENTOS

Eventos en atributos HTML

Hasta el momento veníamos utilizando este tipo de eventos que asociados a los atributos de las etiquetas en HTML, , donde se llama a una función que está en un archivo externo de JS y realiza las instrucciones.

En HTML

```
<button onClick="Saludar()">Saludo</button>
```

En JS

```
let saludar = () => alert("Hola!");
```

Esto funciona, pero puede traer problemas si muchos archivos utilizan el mismo JS y se modifica el nombre o parámetros de la función.

Para que el código sea más legible y fácil de mantener, es preferible no incluir llamadas a funciones Javascript en nuestro código html, y gestionar los eventos desde el archivo externo .js, localizando los elementos del DOM o, mejor aún, gestionar eventos mediante addEventListener().

Eventos en propiedades Javascript .

A una propiedad Javascript le asignaremos la función con el código asociado.

Formato: onclick = function() { ... }

En html

```
<button id="bSaludar">Saludo</button>
```

En JS, dentro de la función que llama el onload.

Seleccionamos el elemento

```
let boton = document.querySelector("#bSaludar");
```

Asociamos a la propiedad onclick, que siempre se escribe en minúsculas la función.

```
boton.onclick = function() {
  alert("Hola!");
}
```

Otro ejemplo similar utilizando setAttribute() y la función saludar().

```
boton.setAttribute("onclick", "saludar()");
```

Si necesitamos agregar más funciones a un evento, podríamos tener una función que ejecute varias funciones, pero más sencillo y legible sería utilizar el método a continuación.

addEventListener()

Agrega una escucha del evento asociándole una función que se ejecutará si dicho evento ocurre. Primer parámetro el evento a escuchar, segundo parámetro la función asociada. Añade una o varias funcionalidades al elemento o puede indicar ciertos comportamientos especiales.

Es mucho más potente y versátil que el resto de los métodos para la mayoría de los casos.

Formato: .addEventListener(evento,función)

```
boton.addEventListener("click", function() {
  alert("Hola!");
});
```

Si lo utilizamos con la función:

```
boton.addEventListener("click", saludar);
```

Múltiples listeners

```
boton.addEventListener("click", saludar);
boton.addEventListener("click", cambiar);
```

Al presionar el botón se efectuarán ambas acciones, ya que hay dos listeners en escucha.

removeEventListener()

Elimina la función previamente asociada a un evento.

Se debe indicar la misma función que añadimos con el addEventListener().

```
let boton = document.querySelector("#bSaludar");
boton.addEventListener("click", saludar)
```

Siendo la función saludar:

```
let saludar = () => {
  alert("Hola!");
  let boton = document.querySelector("#bSaludar");
  boton.removeEventListener("click", saludar,);
};
```

Este ejemplo hace que solo se ejecute una vez el saludo y luego remueve el evento.

objeto Event

Sirve para obtener información relacionada con el evento que se produjo.

Solo es necesario indicar un primer parámetro en la función que dispara el evento y el mismo será de tipo evento con su información asociada.

Ejemplo en JS:

```
let boton = document.querySelector("#bSaludar");
boton.addEventListener("click", (evento) => {
  console.log(evento);
});
```

En la consola muestra toda la información del evento.

```
PointerEvent i
  isTrusted: true
  altKey: false
  altitudeAngle: 1.5707963267948966
  azimuthAngle: 0
  bubbles: true
  button: 0
  buttons: 0
  cancelBubble: false
  cancelable: true
```

El evento contiene una serie de propiedades que dependen del tipo de evento. En el evento click el tipo de evento asociado es PointerEvent donde algunos datos serían:

- type: "click"->Nombre del evento
- pointerType: "mouse" -> Tipo de dispositivo
- altKey: false -> Si la tecla ALT estaba presionada
- ctrlKey: false -> Si la tecla CTRL estaba presionada
- shiftKey: false -> Si la tecla SHIFT estaba presionada
- target: button -> Referencia al elemento que disparó el evento
- clientX: 43 -> Posición en eje X donde se hizo click
- clientY: 16 -> Posición en eje Y donde se hizo click
- detail: 1 -> Contador de veces que se ha hecho click

A todas las propiedades de los eventos se los puede acceder por separado

Algunas propiedades estan presentes en todos los eventos, como las siguientes

- **.type**: Indica el tipo de evento en cuestión.
- **.timeStamp:** Hora en milisegundos en la que se creó el evento.
- **.isTrusted**: Indica si es un evento real de un usuario o uno generado con la función dispatchEvent().

```
boton.addEventListener("click", (evento) => {
  console.log("Tipo de evento: "+evento.type);
  console.log("Hora del evento: "+evento.timeStamp);
  console.log("Evento real: "+evento.isTrusted)
});
```

Método preventDefault()

Para anular el comportamiento y poder cambiar su funcionalidad habitual

Evita que se realice el comportamiento por defecto del evento.

La propiedad **defaultPrevented**, indica si el comportamiento por defecto se ha evitado.

Mediante el método preventDefault() se establecerá la propiedad .defaultPrevented a true y podremos evitar el comportamiento base por defecto de dicho evento y añadirle otro diferente.

Ejemplo: Vamos a realizar validaciones sobre los campos en el formulario y en caso de ser necesario enviar un mensaje y retornar al formulario

En html

```html
<form action="enviar.php" id="formulario">
    <p>
        <label for="nombre">Nombre:</label>
        <input name="nombre" type="text" required>
    </p>
    <p>
        <label for="mensaje">Mensaje:</label>
        <textarea name="mensaje" cols="30" rows="10" required></textarea>
    </p>
    <p>
        <input type="submit" value="Envio">
    </p>
</form>
```

En JS

```javascript
var formulario = document.getElementById('formulario');
formulario.addEventListener('submit', function (evento) {
  evento.preventDefault();
  let nombre = evento.target.nombre.value;
  let mensaje = evento.target.mensaje.value;
  if (nombre.length < 5) {
    alert('El campo nombre debe ser mayor');
    return;
  }
  if (mensaje.trim() == '') {
    alert('El campo mensaje no puede estar vacío');
    return;
  }
  alert('Gracias ' + nombre + ' por enviarnos "' + mensaje + '"');
});
```